



Prédiction de victoire dans League of Legends à partir des statistiques à 15 minutes

Projet de Machine Learning basé sur les données
professionnelles 2024 d'Oracle's Elixir

Réalisé par : Mouad Samsem
Wassim Trachli
Yassine Zaimi
Alae Bajoudi
Saliani Bouchaib

Responsable du module:
Pr. Khadija Lekdioui
Encadré par:
Pr. Said Ammari

Plan de la présentation

1. Introduction

League of Legends et l'esport.

2. Problématique

La question centrale du projet.

3. Dataset et prétraitement

Oracle's Elixir et nettoyage.

4. Feature Engineering

Création des variables clés.

5. EDA

Corrélations et distributions.

6. Modélisation ML

Méthodologie et Algorithmes.

7. Déploiement

Application web interactive Flask.

8. Conclusion

Limites et perspectives.

1. Introduction Générale

- Transformation numérique : Le Machine Learning est central aujourd'hui.
- L'esport : Génération massive de données structurées et objectives en temps réel.
- League of Legends : Un écosystème où la victoire dépend de facteurs quantifiables.
- Objectif : Appliquer des techniques de ML à un environnement compétitif.

Données Compétitives

Contrairement aux sports traditionnels, chaque action génère une métrique précise.

1.1 Le Contexte de League of Legends

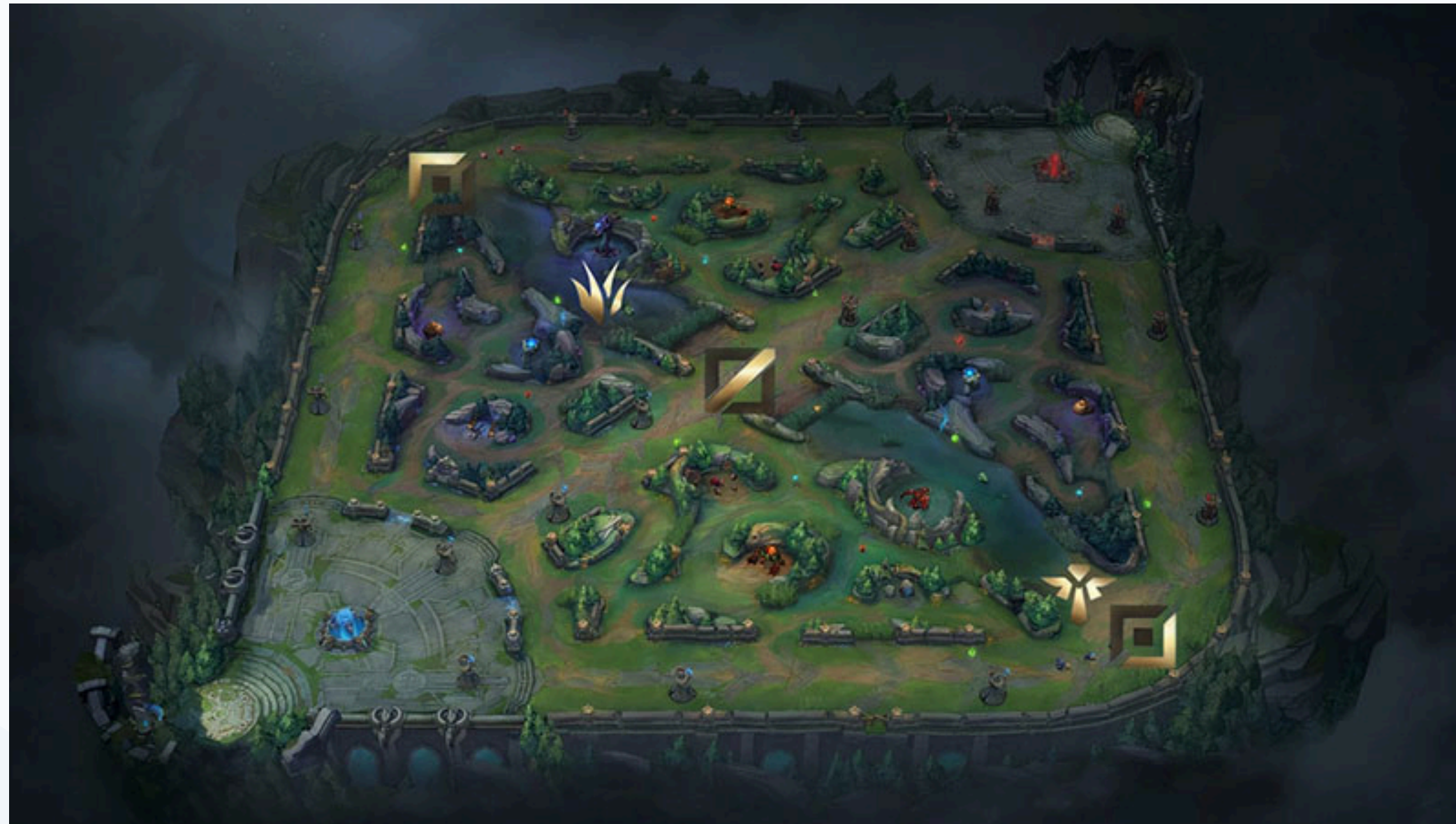
Structure du jeu (MOBA)

Opposition de deux équipes de 5 joueurs sur une carte asymétrique. L'objectif ultime est la destruction du Nexus.

Composante Stratégique

Le succès dépend du contrôle territorial, des objectifs neutres et de l'avantage temporel (Or, Expérience).

1.2 L'Organisation Stratégique



Top

Joueur isolé en haut (duellistes).

Jungle

Évolue en zone neutre.

Mid

Voie centrale, influence rapide.

Bot (ADC)

Dégâts physiques à distance.

Support

Protège l'ADC et contrôle la vision.

1.3 Gestion des Ressources et Draft

- L'Or : Ressource cruciale pour la puissance brute.
- L'Expérience : Fondamentale pour les niveaux.
- Towers & Plates : Pression sur la carte.
- La phase de Draft : Sélection des champions avant le match.

Impact Métier

Quantifier ces variables permet d'évaluer objectivement l'état de la partie à un instant T.

2. Problématique du Projet

"Peut-on prédire l'issue d'un match pro de League of Legends à partir des statistiques disponibles autour de la 15ème minute ?"

Pourquoi 15 minutes ?

C'est le seuil optimal. Avant (draft), l'incertitude est trop grande. Après (late game), le résultat est souvent décidé. À 15 min, les signaux sont forts mais le match reste ouvert.

2.1 Nature du Problème Machine Learning

- Paradigme : Apprentissage Supervisé.
- Type : Classification Binaire.
- Unité d'analyse : L'état d'une équipe pour un match spécifique.

2.2 Objectifs Détaillés

1. Compréhension des données Oracle's Elixir.
2. Préparation et Feature Engineering (variables synthétiques).
3. Modélisation et comparaison rigoureuse d'algorithmes.
4. Sauvegarde du modèle le plus stable.
5. Déploiement via une application web interactive Flask.

3. Source et Structure des Données

Extraction depuis Oracle's Elixir (Saison compétitive 2024).

12 276

Lignes brutes

123

Colonnes

1 023

Matches uniques

Structure d'un match (12 lignes)

5 joueurs Blue Side + 5 joueurs Red Side + 1 agrégation Équipe Blue + 1 agrégation Équipe Red.

3.1 Filtrage et Définition de la Cible

- Filtrage : Conservation des lignes 'team' uniquement.
- Réduction : Passage de 12 276 à 2 046 lignes.

Variable Cible: Result

Classe 1 : Victoire

Classe 0 : Défaite

3.2 Prétraitement et Nettoyage

1. Suppression
des colonnes inutiles (url,
ids,year,split,...).

2. Conversion numérique
et suppression des
valeurs manquantes

3. Split (80/20)
Séparation Train / Test.

Le dataset final contient 1 672 lignes parfaitement équilibrées
(836 victoires, 836 défaites).

4. Feature Engineering (Ressources)

Variable	Description Technique
golddiffat15	Différence d'or entre l'équipe et son adversaire à 15 minutes.
xpdiffat15	Différence d'expérience globale (niveaux et statistiques de base).
platediff	Différence de plaques de tourelles détruites (pression sur la carte).

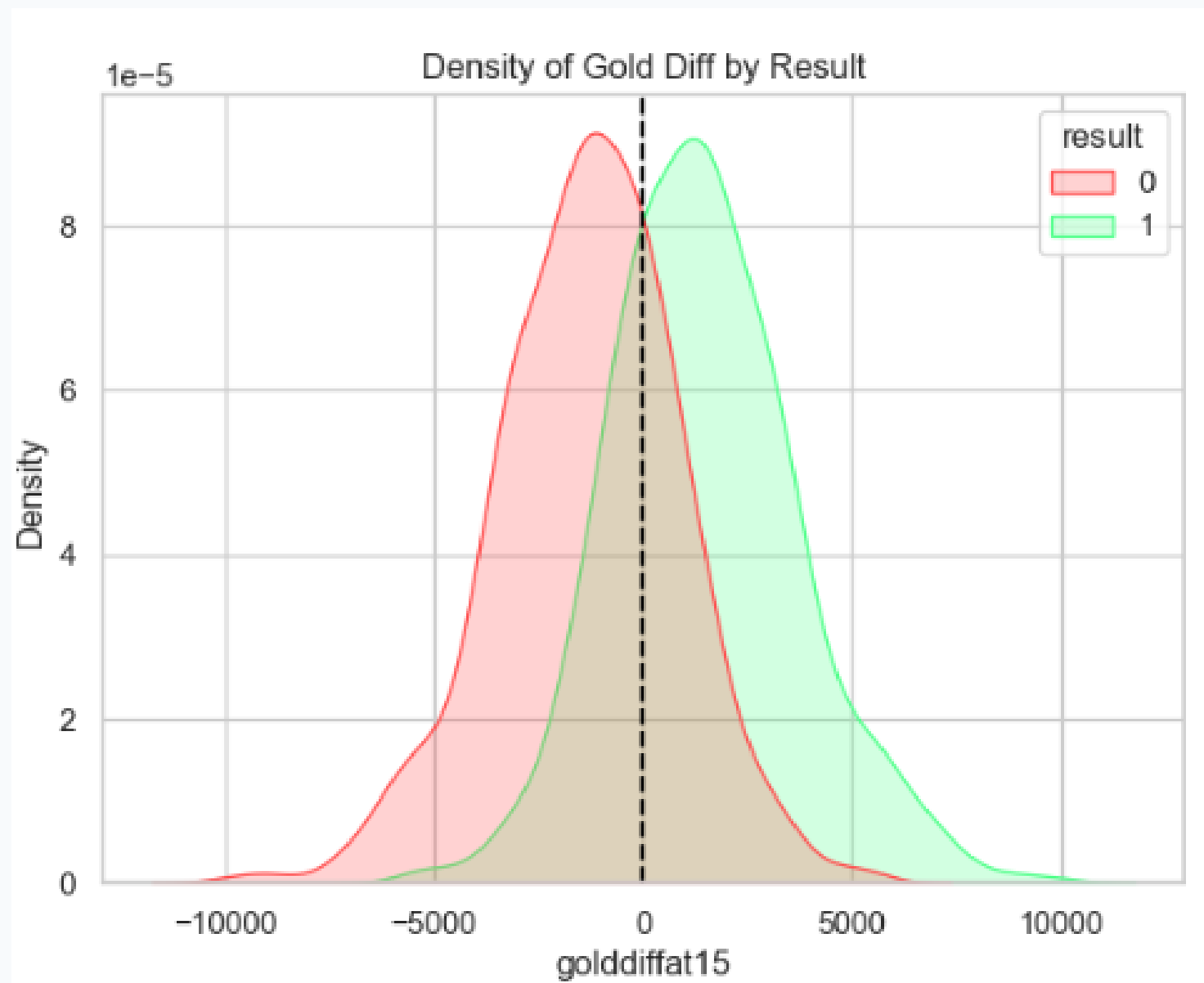
4.1 Feature Engineering (Combats & Objectifs)

Variable	Description Technique
kda_15	Efficacité collective : $(\text{kills} + \text{assists}) / (\text{deaths} + 1)$.
first[x]	Variables binaires : firstblood, firstdragon, firstherald, firsttower.
side_encoded	Encodage du côté de la carte (Blue = 1, Red = 0).

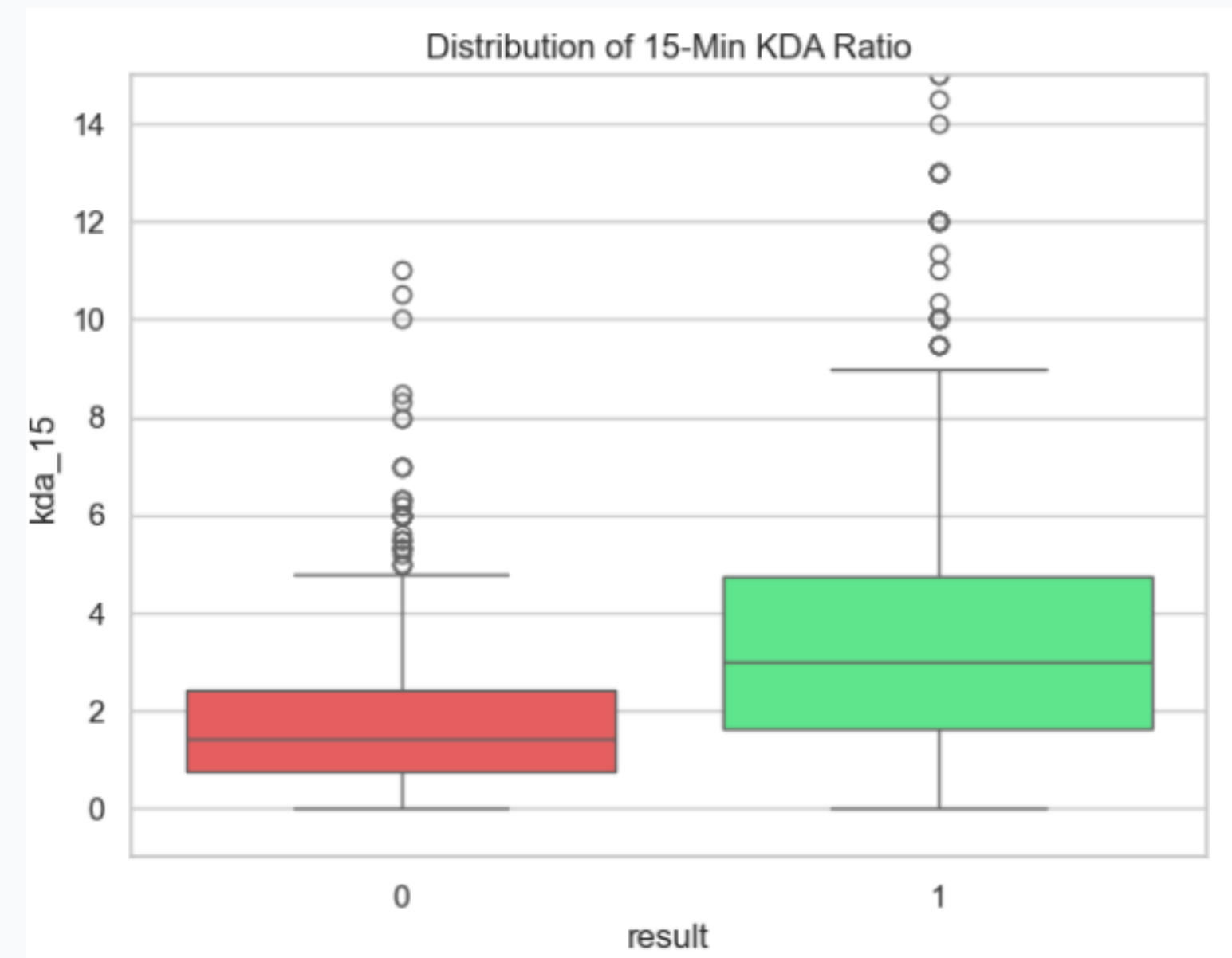
4.2 Feature Engineering (Draft)

- Création de la variable `draft_adv`.
- Objectif : Quantifier mathématiquement l'avantage théorique de la composition.
- Méthode : Différence entre la moyenne des taux de victoire historiques (win rates) des 5 champions alliés et celle des adversaires.

5. Analyse Exploratoire: Ressources et combat

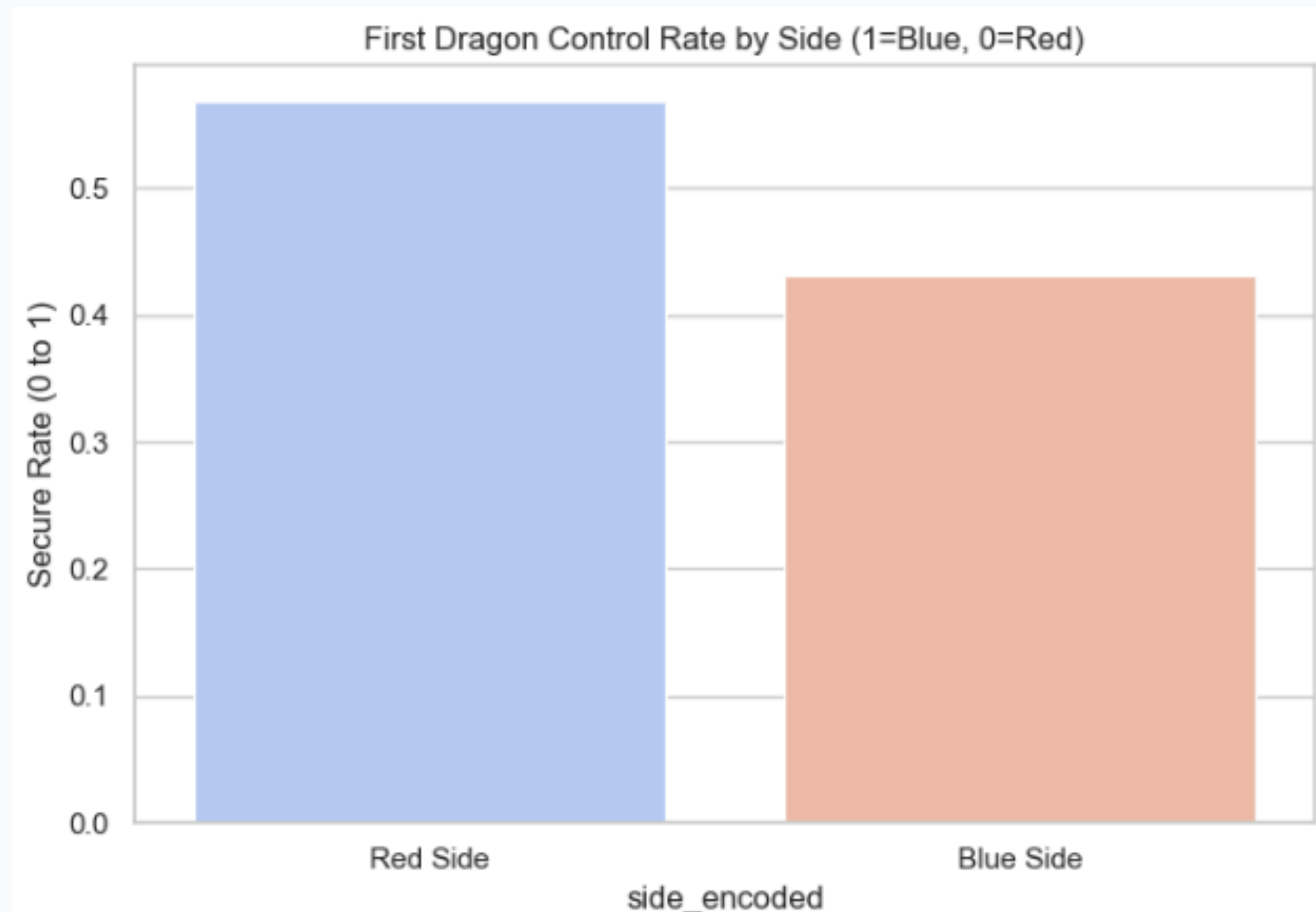


Distribution de la différence d'or à 15 minutes
selon l'issue du match

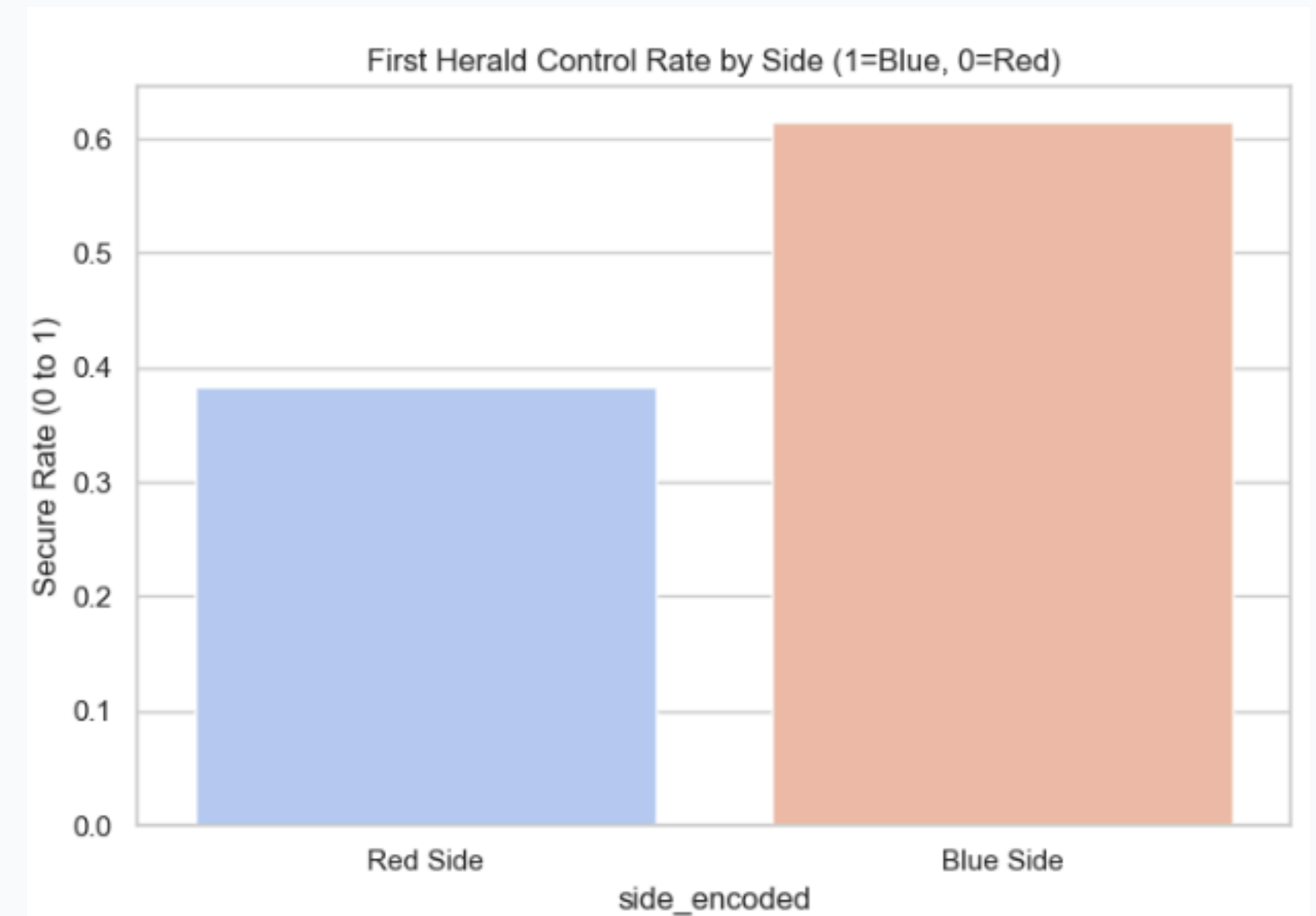


Analyse comparative du ratio KDA à 15 minutes
par résultat

5.1 Analyse Exploratoire: objectifs initiaux



Taux de sécurisation du Premier Dragon selon le côté de la carte



Taux de sécurisation du Premier Herald selon le côté de la carte

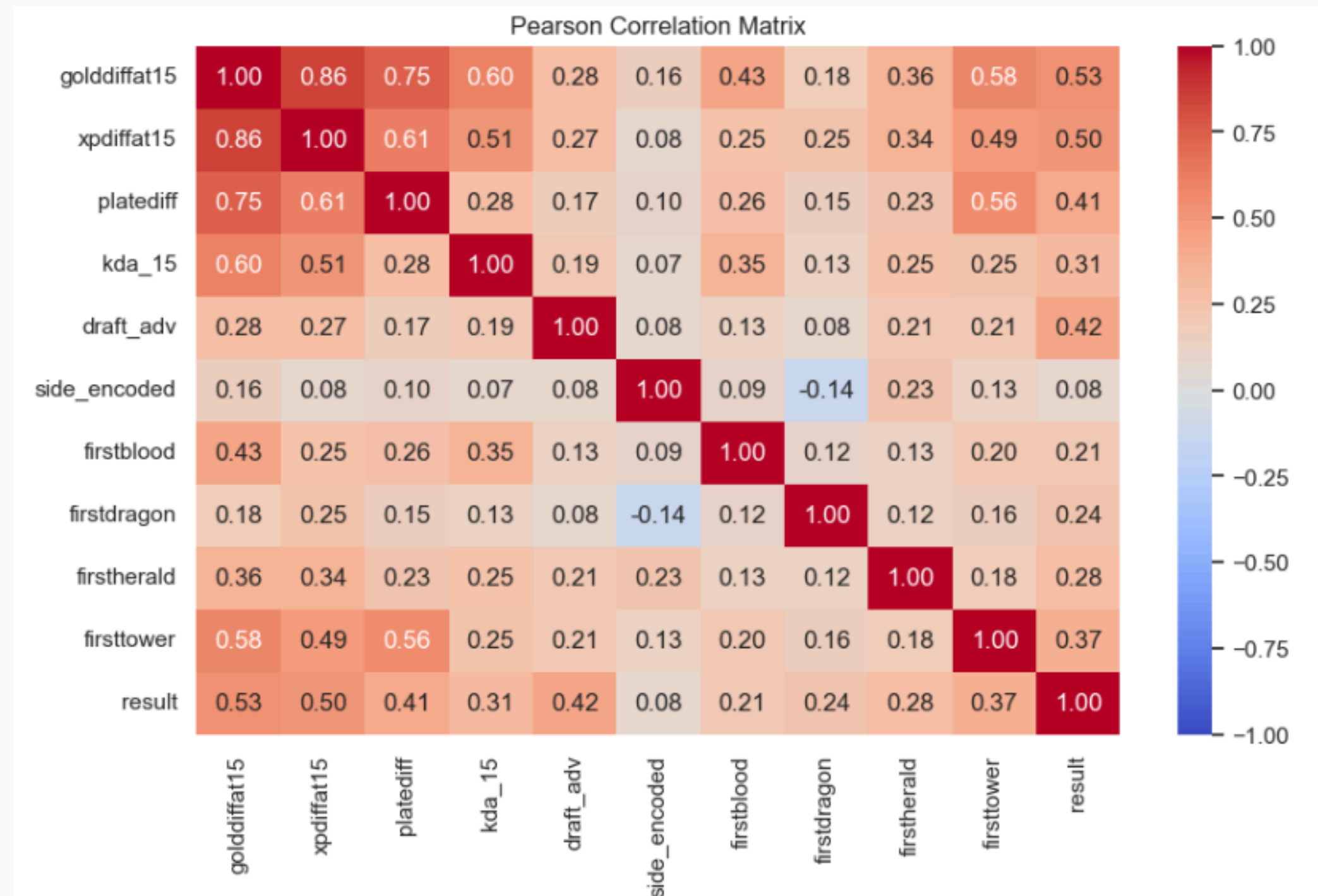
5.2 Analyse Exploratoire : Corrélations

Corrélation avec la variable résultat :

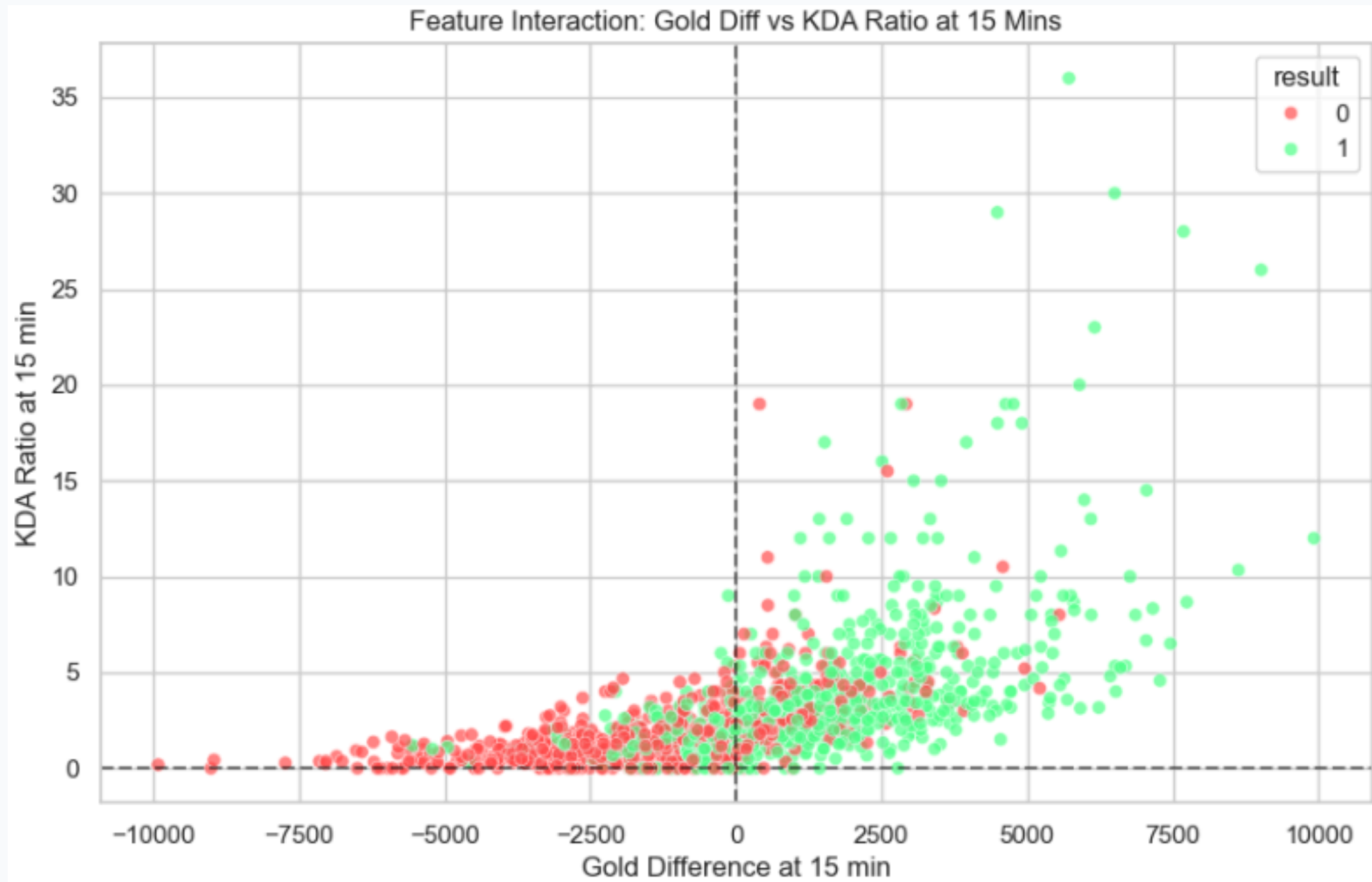
- golddiffat15 : 0.530
- xpdiffat15 : 0.498
- draft_adv : 0.420
- platediff : 0.406
- side_encoded : 0.077 (Impact très faible)

Interprétation:

L'économie et l'expérience dominant. La stratégie (Draft) est aussi déterminante que les objectifs.



5.3 Analyse Exploratoire : Incertitude



26.47%
Taux de Comeback

4 Cas
Throws(>4000g d'avance)

Sur 665 équipes en retard économique à 15 minutes, 176 ont fini par gagner.
L'avance matérielle ne dicte pas une règle absolue.

| 6. Méthodologie Machine Learning

- Pipelines Scikit-learn : Regroupement normalisation + modèle.
- Normalisation : RobustScaler (gestion des outliers d'or/XP).
- Évaluation : Accuracy, Precision, Recall, F1-Score et CV (5-folds).

| 6.1 Algorithmes Utilisés

Linéaire
Logistic Regression

Arbres
Random Forest

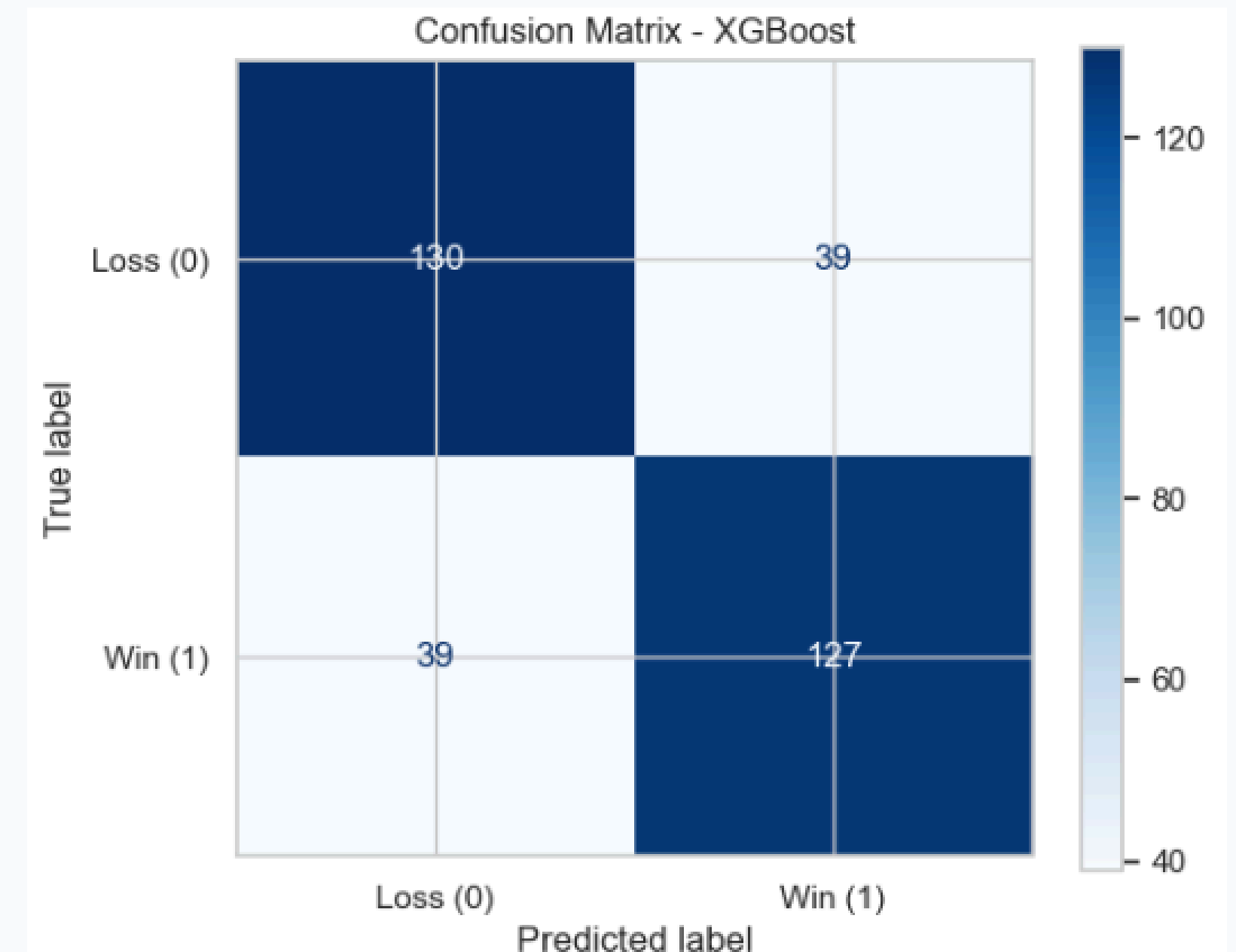
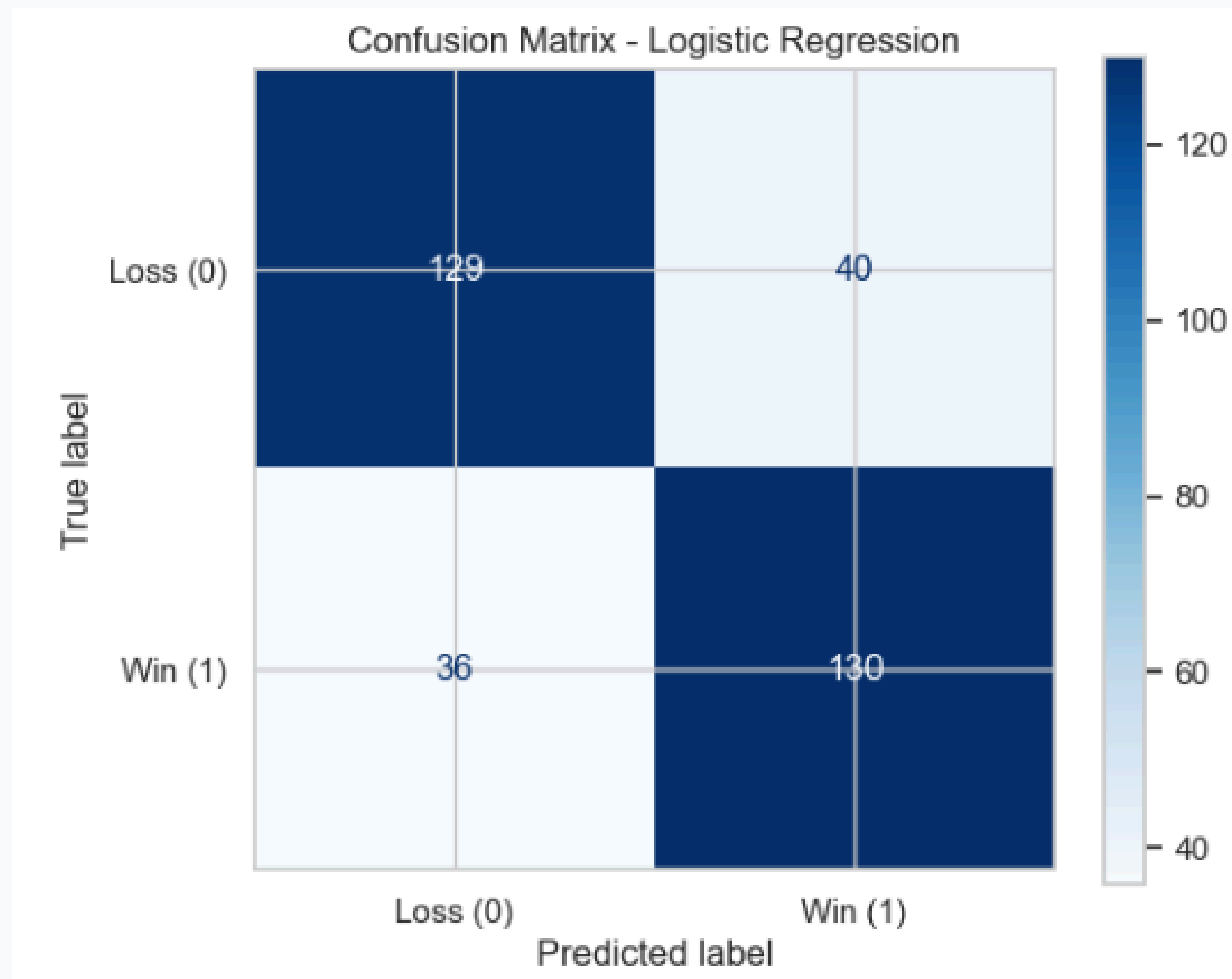
Boosting
Gradient, XGBoost,
Adaboost

Modèles d'ensemble
Voting, Stacking
classifier

6.2 Résultats des Modèles

Modèle	Résultat en Accuracy
Random Forest	77.91% (F1: 77.98%)
Logistic Regression	77.31% (F1: 77.38%)
XGBoost	76.72% (F1: 76.51%)
AdaBoost	75.22% (F1: 74.77%)
Gradient Boosting	74.33%(F1: 73.78%)

6.3 Matrice de confusion



6.4 Optimisation des hyperparamètres et validation croisée

- GridSearchCV pour Random forest et Logistic Regression
- RandomizedSearchCV pour XGBoost

Modèle	Meilleurs paramètres	Score CV
Random Forest	<code>n_estimators=50,</code> <code>max_features=sqrt,</code> <code>min_samples_split=2</code>	<code>max_depth=8,</code> 0.7651
XGBoost	<code>n_estimators=100,</code> <code>learning_rate=0.05,</code> <code>colsample_bytree=0.7</code>	<code>max_depth=3,</code> 0.7696 <code>subsample=0.9,</code>
Logistic Regression	<code>C=0.1, penalty=l2, solver=liblinear</code>	0.7726

6.5 Modèles d'ensemble et résultats

Le Voting et Stacking combinent ces trois familles de modèles pour maximiser la robustesse:

- Régression Logistique (Modèle Linéaire) : Fournit une base probabiliste stable et une frontière de décision géométrique globale
- Random Forest (Ensemble par Bagging) : Réduit drastiquement la variance en moyennant des arbres de décision indépendants entraînés en parallèle.
- XGBoost (Ensemble par Boosting) : Réduit drastiquement le biais en corrigeant séquentiellement et de manière adaptative les erreurs des arbres précédents.

Résultats:

Modèle	Accuracy
Voting Classifier	0.7761
Stacking Classifier	0.7791

Après validation croisée:

Modèle	Moyenne CV	Écart-type
Voting Classifier	0.7711	0.0320
Stacking Classifier	0.7719	0.0323

6.6 Modèles Finaux & Importances

Voting Classifier (Retenu)

Accuracy : 77.61%

Choisi pour son compromis optimal entre stabilité et simplicité de déploiement.

Top 3 Features

- 1. Différence d'Or (26.5%)**
- 2. Différence d'XP (23.6%)**
- 3. Advantage Draft (20.8%)**

7. Déploiement : Application Flask

- Objectif : Rendre le modèle utilisable (analyste, commentateur).
- Architecture : Backend app.py, Modèle voting_model.pkl, Interface HTML.
- Processus : Saisie de l'état du match → Calcul des 10 variables → Inférence → Probabilité.

8. Limites Actuelles Identifiées

Data Leakage
Split aléatoire par ligne.

Timing
Objectifs potentiellement
post-15min.

Draft
Win rates historiques
trop simples.

Approximation de l'XP
via les niveaux.

| 8.1 Roadmap d'Amélioration

1. Validation Robuste : Implémenter GroupShuffleSplit.
2. Filtrage Temporel : Assainir les datasets ($\leq 15:00$).
3. Lissage Statistique : Appliquer des méthodes bayésiennes aux taux de victoire.
4. Refonte UX Flask : Corriger la logique asymétrique Blue/Red.

| 8.2 Conclusion Générale

Ce projet valide un cycle complet de Machine Learning appliqué à l'esport en prouvant la haute prédictibilité d'un match de League of Legends à 15 minutes.

Le feature engineering démontre que la macro-économie (Or : 26,5 % importance, Expérience : 23,6 %) domine largement le signal prédictif, bien que le taux de comeback de 26,47 % confirme la non-linéarité des parties et la nécessité d'une approche par IA.

Au niveau des résultats, malgré la performance maximale de 77,91 % d'accuracy atteinte par le Random Forest et le Stacking, le Voting Classifieur (77,61 %) a été préféré pour l'application Flask en raison de sa stabilité et de sa robustesse.

Enfin, l'analyse critique ouvre la voie à des correctifs prioritaires : l'intégration d'un split GroupShuffleSplit par match pour éliminer le data leakage et un contrôle strict des timestamp des objectifs.

**MERCI
POUR VOTRE
ATTENTION**